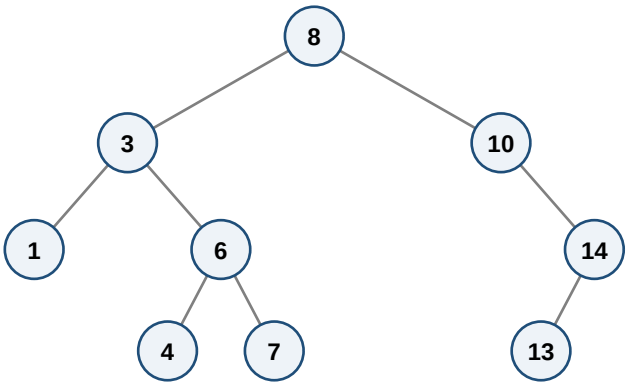
Binary Search Tree

Placement notes for the 24-problem BST checklist - idea, approach, C++ code, complexity, pitfalls

#	Problem	LC	Level	Core pattern	Done
1	Inorder Traversal	94	Easy	Iterative inorder (stack)	[ ]
2	Search in BST	700	Easy	Walk one path	[ ]
3	Insert into BST	701	Medium	Walk + relink	[ ]
4	Minimum Absolute Difference in BST	530	Easy	Inorder + prev	[ ]
5	Range Sum of BST	938	Easy	Prune by range	[ ]
6	Validate BST	98	Medium	Range (lo, hi)	[ ]
7	LCA of a BST	235	Medium	Split point	[ ]
8	Two Sum IV - Input is a BST	653	Easy	Inorder + 2 pointers	[ ]
9	Trim a BST	669	Medium	Prune + relink	[ ]
10	Closest Nodes Queries in a BST	2476	Medium	Inorder + binary search	[ ]
11	Kth Smallest Element in a BST	230	Medium	Inorder, stop at k	[ ]
12	Delete Node in a BST	450	Medium	3 cases + successor	[ ]
13	Convert Sorted Array to BST	108	Easy	Middle = root	[ ]
14	Balance a BST	1382	Medium	Inorder + rebuild	[ ]
15	BST Iterator	173	Medium	Paused inorder	[ ]
16	Inorder Successor in BST	285	Medium	Walk + candidate	[ ]
17	Kth Largest Element in a Stream	703	Easy	Min-heap size k	[ ]
18	Construct BST from Preorder	1008	Medium	Upper bound recursion	[ ]
19	Recover BST	99	Medium	Inorder drops	[ ]
20	Convert BST to Greater Tree	538	Medium	Reverse inorder	[ ]
21	Unique Binary Search Trees	96	Medium	Catalan DP	[ ]
22	Unique Binary Search Trees II	95	Medium	Recursion over roots	[ ]
23	Maximum Sum BST in Binary Tree	1373	Hard	Postorder info	[ ]
24	Morris Traversal (practice)	-	Concept	Threaded tree, O(1)	[ ]

BST Fundamentals (read before solving)

BST property: for every node, all values in the left subtree are smaller and all values in the right subtree are larger (LeetCode: strictly, no duplicates unless stated).



Inorder of the tree above: `1 3 4 6 7 8 10 13 14` - always sorted.

Core facts

- **Inorder of a BST is sorted.** At least 10 of the 24 problems reduce to this one line.
- Search / insert / delete cost **O(h)**: O(log n) if balanced, O(n) if skewed (e.g. inserting sorted data).
- Min = leftmost node, Max = rightmost node.
- Inorder successor = leftmost node of right subtree (if right exists), else nearest ancestor where you went left.
- Preorder alone (or postorder alone) uniquely determines a BST. Inorder alone does NOT.
- Self-balancing BSTs: AVL (strict, faster lookups), Red-Black (looser, faster inserts; used by C++ `std::set`/`std::map`, Java `TreeMap`).
- Number of structurally unique BSTs with n keys = Catalan(n).

TreeNode used in all code

```
struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
    TreeNode(int x, TreeNode* l, TreeNode* r) : val(x), left(l), right(r) {}
};
```

The 6 templates that solve everything here

Template	Problems
1. Walk down one path (go left/right by comparison)	700, 701, 235, 285, floor/ceil
2. Inorder = sorted array (+ prev pointer / two pointers / binary search)	94, 530, 98, 653, 2476, 230, 99, 1382
3. Pass range (lo, hi) or bound downward	98, 938, 669, 1008
4. Return node to re-link parent (root->left = f(root->left))	701, 450, 669, 108, 1382
5. Postorder - return info bottom-up	1373, Largest BST, Balanced check
6. Stack-based / threaded inorder	173, 230, 653 follow-up, Morris

Problems

1. Inorder Traversal LC 94 [Easy]

Key idea

Inorder = Left, Node, Right. For a BST this visits values in **sorted (ascending) order** - the single most useful BST fact.

Approach

- Recursive version is 3 lines; interviewers usually ask for the **iterative** one.
- Iterative: push all left children onto a stack, pop one, record it, then move to its right child and repeat.

Code (C++)

```
vector<int> inorderTraversal(TreeNode* root) {
    vector<int> res;
    stack<TreeNode*> st;
    TreeNode* cur = root;
    while (cur || !st.empty()) {
        while (cur) { st.push(cur); cur = cur->left; }  // go left fully
        cur = st.top(); st.pop();
        res.push_back(cur->val);                          // visit
        cur = cur->right;                                 // then right
    }
    return res;
}
```

Complexity: Time O(n), Space O(h) (h = height; O(log n) balanced, O(n) skewed).

Pitfalls / follow-ups

- Loop condition is `cur || !st.empty()` - forgetting `cur` exits too early.
- Know all three orders: Pre (N L R), In (L N R), Post (L R N). O(1)-space version = Morris (see #24).

2. Search in BST LC 700 [Easy]

Key idea

At each node, the BST property tells you which **one** side can contain the target - discard the other half.

Approach

- If `val < node->val` go left, if greater go right, if equal return node.

Code (C++)

```
TreeNode* searchBST(TreeNode* root, int val) {
    while (root && root->val != val)
        root = (val < root->val) ? root->left : root->right;
    return root;   // nullptr if not found
}
```

Complexity: Time O(h), Space O(1) iterative (O(h) if recursive).

Pitfalls / follow-ups

- Prefer iterative - no recursion stack. This 'walk down one path' template is reused in 235, 285, 701, 2476.

3. Insert into BST LC 701 [Medium]

Key idea

Walk down like search; the new value always becomes a **new leaf** at the null position where search ends.

Approach

- Recursive trick: `root->left = insert(root->left, val)` re-links the tree automatically.
- Base case: empty subtree -> return a new node.

Code (C++)

```
TreeNode* insertIntoBST(TreeNode* root, int val) {
    if (!root) return new TreeNode(val);
    if (val < root->val) root->left  = insertIntoBST(root->left, val);
    else                 root->right = insertIntoBST(root->right, val);
    return root;
}
```

Complexity: Time O(h), Space O(h) recursion (O(1) iterative with a parent pointer).

Pitfalls / follow-ups

- Never rebalance or rotate unless asked - any valid BST is accepted.
- Inserting sorted data one by one produces a skewed tree (h = n) - that is why balanced trees exist.

4. Minimum Absolute Difference in BST LC 530 [Easy]

Key idea

Inorder is sorted, so the minimum difference must be between **adjacent** values in inorder. Track the previous node.

Approach

- Do an inorder traversal keeping a `prev` pointer; answer = min of `cur - prev` over all steps.
- Same problem as LC 783 (Minimum Distance Between BST Nodes).

Code (C++)

```
int ans = INT_MAX;
TreeNode* prev = nullptr;
void inorder(TreeNode* node) {
    if (!node) return;
    inorder(node->left);
    if (prev) ans = min(ans, node->val - prev->val);
    prev = node;
    inorder(node->right);
}
int getMinimumDifference(TreeNode* root) { inorder(root); return ans; }
```

Complexity: Time O(n), Space O(h).

Pitfalls / follow-ups

- **'prev in inorder' pattern** - also used in 98 (validate), 99 (recover). Learn it once.
- No need for abs(): inorder is increasing so `cur - prev` is already positive.

5. Range Sum of BST LC 938 [Easy]

Key idea

Sum values in [low, high]. Use the BST property to **prune** subtrees that cannot be in range.

Approach

- node < low -> whole left subtree is smaller too, only go right.
- node > high -> only go left.
- Otherwise count node and recurse both sides.

Code (C++)

```
int rangeSumBST(TreeNode* root, int low, int high) {
    if (!root) return 0;
    if (root->val < low)  return rangeSumBST(root->right, low, high);
    if (root->val > high) return rangeSumBST(root->left,  low, high);
    return root->val + rangeSumBST(root->left, low, high)
                     + rangeSumBST(root->right, low, high);
}
```

Complexity: Time O(n) worst, but visits only the nodes near the range in practice. Space O(h).

Pitfalls / follow-ups

- The same three-case pruning shape appears in 669 (Trim BST).

6. Validate BST LC 98 [Medium]

Key idea

Every node must lie in an **allowed range (lo, hi)** inherited from its ancestors - checking only the direct children is WRONG.

Approach

- Going left: upper bound becomes node->val. Going right: lower bound becomes node->val.
- Alternative: inorder traversal must be **strictly increasing** (prev pattern).

Code (C++)

```
bool check(TreeNode* node, long long lo, long long hi) {
    if (!node) return true;
    if (node->val <= lo || node->val >= hi) return false;
    return check(node->left, lo, node->val) &&
           check(node->right, node->val, hi);
}
bool isValidBST(TreeNode* root) { return check(root, LLONG_MIN, LLONG_MAX); }
```

Complexity: Time O(n), Space O(h).

Pitfalls / follow-ups

- Classic trap: tree `5 -> (4, 6 -> (3, 7))` - each parent/child pair looks fine but 3 is in 5's right subtree.
- Use `long long` bounds (or nullptr bounds) - node values can be INT_MIN / INT_MAX.
- Duplicates: LeetCode wants strict `<`, so equal values make it invalid.

7. LCA of a BST LC 235 [Medium]

Key idea

The LCA is the **split point**: the first node where p and q go to different sides (or one of them equals the node).

Approach

- Both smaller -> go left. Both larger -> go right. Else current node is the answer.

Code (C++)

```
TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
    while (root) {
        if (p->val < root->val && q->val < root->val)      root = root->left;
        else if (p->val > root->val && q->val > root->val) root = root->right;
        else return root;   // split point
    }
    return nullptr;
}
```

Complexity: Time O(h), Space O(1).

Pitfalls / follow-ups

- Compare with LC 236 (LCA of a general binary tree) which needs full DFS in O(n). Interviewers love asking the difference.

8. Two Sum IV - Input is a BST LC 653 [Easy]

Key idea

Inorder gives a sorted array -> classic **two pointers**. (Or DFS with an unordered_set like normal Two Sum.)

Approach

- Approach A: inorder into vector, then i = 0, j = n-1 two-pointer scan.
- Approach B: DFS, for each node check if `k - val` is in a hash set, then insert val.

Code (C++)

```
void inorder(TreeNode* n, vector<int>& v) {
    if (!n) return;
    inorder(n->left, v); v.push_back(n->val); inorder(n->right, v);
}
bool findTarget(TreeNode* root, int k) {
    vector<int> v; inorder(root, v);
    int i = 0, j = (int)v.size() - 1;
    while (i < j) {
        int s = v[i] + v[j];
        if (s == k) return true;
        if (s < k) i++; else j--;
    }
    return false;
}
```

Complexity: Time O(n), Space O(n).

Pitfalls / follow-ups

- **Follow-up**: O(h) space - use two BST iterators (#15), one normal (next smallest) and one reversed (next largest), as the two pointers.

9. Trim a BST LC 669 [Medium]

Key idea

Remove nodes outside [low, high] while keeping the BST valid. If a node is out of range, its **entire one side** is out too, so return the trimmed other side.

Approach

- node < low -> discard node + left subtree, return trim(right).
- node > high -> discard node + right subtree, return trim(left).
- In range -> trim both children and keep node.

Code (C++)

```
TreeNode* trimBST(TreeNode* root, int low, int high) {
    if (!root) return nullptr;
    if (root->val < low)  return trimBST(root->right, low, high);
    if (root->val > high) return trimBST(root->left,  low, high);
    root->left  = trimBST(root->left,  low, high);
    root->right = trimBST(root->right, low, high);
    return root;
}
```

Complexity: Time O(n), Space O(h).

Pitfalls / follow-ups

- Same skeleton as 938. Pattern: 'return the recursive result to re-link the parent' (like 701, 450).

10. Closest Nodes Queries in a BST LC 2476 [Medium]

Key idea

For each query q return [largest value <= q, smallest value >= q] (or -1). The tree can be **unbalanced** (up to 1e5 nodes), so walking the tree per query can TLE -> convert to a sorted array once and binary search.

Approach

- Inorder -> sorted vector `v`.
- `it = lower_bound(v, q)` gives smallest >= q (ceil).
- Floor = q itself if `*it == q`, else the element before `it`.

Code (C++)

```
vector<vector<int>> closestNodes(TreeNode* root, vector<int>& queries) {
    vector<int> v; inorder(root, v);          // sorted values
    vector<vector<int>> res;
    for (int q : queries) {
        auto it = lower_bound(v.begin(), v.end(), q);
        int maxi = (it != v.end()) ? *it : -1;
        int mini;
        if (it != v.end() && *it == q) mini = q;
        else mini = (it != v.begin()) ? *prev(it) : -1;
        res.push_back({mini, maxi});
    }
    return res;
}
```

Complexity: Time O(n + Q log n), Space O(n).

Pitfalls / follow-ups

- Floor / ceil in a BST directly (O(h)) is a common GFG question - same walk as #16 (successor).
- Lesson: a BST only gives O(log n) if it is balanced; check constraints.

11. Kth Smallest Element in a BST LC 230 [Medium]

Key idea

Kth smallest = kth node visited in inorder. Use iterative inorder and **stop early** at k.

Approach

- Decrement k each time a node is popped; return when k hits 0.

Code (C++)

```
int kthSmallest(TreeNode* root, int k) {
    stack<TreeNode*> st;
    TreeNode* cur = root;
    while (cur || !st.empty()) {
        while (cur) { st.push(cur); cur = cur->left; }
        cur = st.top(); st.pop();
        if (--k == 0) return cur->val;
        cur = cur->right;
    }
    return -1;
}
```

Complexity: Time O(h + k), Space O(h).

Pitfalls / follow-ups

- **Follow-up** (tree modified often, many queries): store `leftCount` (size of left subtree) in each node -> O(h) per query, like an order-statistic tree.
- Kth largest = reverse inorder (Right, Node, Left).

12. Delete Node in a BST LC 450 [Medium]

Key idea

Find the node, then 3 cases: **leaf**, **one child**, **two children**.

Approach

- 0 or 1 child: return the other child to the parent (covers leaf too).
- 2 children: copy the **inorder successor** (min of right subtree) into the node, then delete that successor from the right subtree.
- (Inorder predecessor = max of left subtree works equally.)

Code (C++)

```
TreeNode* deleteNode(TreeNode* root, int key) {
    if (!root) return nullptr;
    if (key < root->val)      root->left  = deleteNode(root->left, key);
    else if (key > root->val) root->right = deleteNode(root->right, key);
    else {
        if (!root->left)  return root->right;   // 0 or 1 child
        if (!root->right) return root->left;
        TreeNode* succ = root->right;             // 2 children
        while (succ->left) succ = succ->left;
        root->val = succ->val;
        root->right = deleteNode(root->right, succ->val);
    }
    return root;
}
```

Complexity: Time O(h), Space O(h).

Pitfalls / follow-ups

- Most-asked BST question in interviews. Be able to explain why the successor has at most one (right) child.
- In real C++ code, `delete` the removed node to avoid leaks (fine to skip on LeetCode).

13. Convert Sorted Array to BST LC 108 [Easy]

Key idea

Pick the **middle** element as root so both halves have equal size -> height-balanced. Recurse on halves.

Approach

- build(l, r): if l > r return null; m = mid; root = a[m]; left = build(l, m-1); right = build(m+1, r).

Code (C++)

```
TreeNode* build(vector<int>& a, int l, int r) {
    if (l > r) return nullptr;
    int m = l + (r - l) / 2;
    TreeNode* node = new TreeNode(a[m]);
    node->left  = build(a, l, m - 1);
    node->right = build(a, m + 1, r);
    return node;
}
TreeNode* sortedArrayToBST(vector<int>& nums) {
    return build(nums, 0, (int)nums.size() - 1);
}
```

Complexity: Time O(n), Space O(log n) recursion.

Pitfalls / follow-ups

- Sorted **linked list** version (LC 109): find middle with slow/fast pointers, or simulate inorder in O(n).

14. Balance a BST LC 1382 [Medium]

Key idea

Inorder (sorted) + #108. Flatten the BST into a sorted list, then rebuild by picking middles.

Approach

- Store node pointers (not values) to reuse the existing nodes - no new allocations.

Code (C++)

```
void inorder(TreeNode* n, vector<TreeNode*>& v) {
    if (!n) return;
    inorder(n->left, v); v.push_back(n); inorder(n->right, v);
}
TreeNode* build(vector<TreeNode*>& v, int l, int r) {
    if (l > r) return nullptr;
    int m = l + (r - l) / 2;
    TreeNode* node = v[m];
    node->left  = build(v, l, m - 1);
    node->right = build(v, m + 1, r);
    return node;
}
TreeNode* balanceBST(TreeNode* root) {
    vector<TreeNode*> v; inorder(root, v);
    return build(v, 0, (int)v.size() - 1);
}
```

Complexity: Time O(n), Space O(n).

Pitfalls / follow-ups

- Theory question: self-balancing trees (AVL, Red-Black) keep h = O(log n) using rotations. `std::set` / `std::map` are Red-Black trees.
- Advanced O(1)-space alternative: Day-Stout-Warren (DSW) algorithm - just know the name.

15. BST Iterator LC 173 [Medium]

Key idea

Iterative inorder, but **paused**. The stack holds the path of pending ancestors; `next()` pops one and pushes the left spine of its right child.

Approach

- Constructor: push left spine of root.
- next(): pop top, push left spine of top->right, return top->val.
- hasNext(): stack not empty.

Code (C++)

```
class BSTIterator {
    stack<TreeNode*> st;
    void pushLeft(TreeNode* n) { while (n) { st.push(n); n = n->left; } }
public:
    BSTIterator(TreeNode* root) { pushLeft(root); }
    int next() {
        TreeNode* n = st.top(); st.pop();
        pushLeft(n->right);
        return n->val;
    }
    bool hasNext() { return !st.empty(); }
};
```

Complexity: next() amortized O(1) (each node pushed/popped once), Space O(h).

Pitfalls / follow-ups

- Interviewers check you know **why** it is amortized O(1), not O(h) per call.
- Reverse iterator (next largest) = same with right/left swapped -> used in 653 follow-up.

16. Inorder Successor in BST LC 285 [Medium]

Key idea

Successor = smallest value **greater than** p. Walk from root: whenever you go left, the current node is a candidate.

Approach

- p->val < node->val: node is a candidate, go left to find a smaller one.
- Otherwise go right.
- Predecessor is the mirror (candidate when going right).

Code (C++)

```
TreeNode* inorderSuccessor(TreeNode* root, TreeNode* p) {
    TreeNode* succ = nullptr;
    while (root) {
        if (p->val < root->val) { succ = root; root = root->left; }
        else root = root->right;
    }
    return succ;
}
// Predecessor:
// if (p->val > root->val) { pred = root; root = root->right; } else root = root->left;
```

Complexity: Time O(h), Space O(1).

Pitfalls / follow-ups

- Alternate (pointer-based): if p has a right child, successor = leftmost of p->right; else it is the nearest ancestor for which p is in the left subtree.
- LC 285 is premium - practice on GFG 'Inorder Successor in BST'. With parent pointers: LC 510.

17. Kth Largest Element in a Stream LC 703 [Easy]

Key idea

Keep a **min-heap of size k** holding the k largest elements seen so far. Its top is the kth largest.

Approach

- add(val): push, if size > k pop the smallest; return top.
- BST connection: a balanced BST / `multiset` or an augmented BST with subtree counts also works.

Code (C++)

```
class KthLargest {
    priority_queue<int, vector<int>, greater<int>> pq;   // min-heap
    int k;
public:
    KthLargest(int k, vector<int>& nums) : k(k) {
        for (int x : nums) add(x);
    }
    int add(int val) {
        pq.push(val);
        if ((int)pq.size() > k) pq.pop();
        return pq.top();
    }
};
```

Complexity: add() O(log k), Space O(k).

Pitfalls / follow-ups

- Remember: kth **largest** -> **min**-heap of size k; kth **smallest** -> **max**-heap of size k.

18. Construct BST from Preorder LC 1008 [Medium]

Key idea

Preorder is Node, Left, Right. Consume values with a global index and an **upper bound**: a value belongs in the current subtree only if it is below the bound.

Approach

- build(bound): if next value > bound return null; create node; left = build(node->val); right = build(bound).
- Each value is consumed exactly once -> O(n).

Code (C++)

```
int idx = 0;
TreeNode* build(vector<int>& pre, int bound) {
    if (idx == (int)pre.size() || pre[idx] > bound) return nullptr;
    TreeNode* node = new TreeNode(pre[idx++]);
    node->left  = build(pre, node->val);
    node->right = build(pre, bound);
    return node;
}
TreeNode* bstFromPreorder(vector<int>& preorder) {
    return build(preorder, INT_MAX);
}
```

Complexity: Time O(n), Space O(h).

Pitfalls / follow-ups

- Simpler alternative: insert each value with #701 -> O(n log n) avg, O(n^2) worst.
- Another: sort preorder to get inorder, then use the standard pre+in construction (LC 105).
- For a BST, preorder alone (or postorder alone) is enough to rebuild it uniquely.

19. Recover BST LC 99 [Medium]

Key idea

Two nodes were swapped. In inorder, look for **drops** (prev > cur). First drop -> `first = prev`; last drop -> `second = cur`. Swap their values.

Approach

- Adjacent swap -> only one drop (first = prev, second = cur of that drop).
- Non-adjacent swap -> two drops (first from 1st drop, second from 2nd drop).
- Example inorder 1 2 **6** 4 5 **3** 7 -> drops at (6,4) and (5,3) -> swap 6 and 3.

Code (C++)

```
TreeNode *first = nullptr, *second = nullptr, *prv = nullptr;
void inorder(TreeNode* n) {
    if (!n) return;
    inorder(n->left);
    if (prv && prv->val > n->val) {
        if (!first) first = prv;
        second = n;
    }
    prv = n;
    inorder(n->right);
}
void recoverTree(TreeNode* root) {
    inorder(root);
    swap(first->val, second->val);
}
```

Complexity: Time O(n), Space O(h). **O(1) space** follow-up -> do the same check inside Morris traversal (#24).

Pitfalls / follow-ups

- Always update `second` on every drop - that handles both cases with one code path.

20. Convert BST to Greater Tree LC 538 [Medium]

Key idea

Each node becomes val + sum of all greater values. Visit in **reverse inorder** (Right, Node, Left) keeping a running sum.

Approach

- Same as LC 1038 (BST to Greater Sum Tree).

Code (C++)

```
int running = 0;
TreeNode* convertBST(TreeNode* root) {
    if (!root) return nullptr;
    convertBST(root->right);
    running += root->val;
    root->val = running;
    convertBST(root->left);
    return root;
}
```

Complexity: Time O(n), Space O(h).

Pitfalls / follow-ups

- Reverse inorder = descending order. Useful for any 'sum/count of greater elements' question.

21. Unique Binary Search Trees LC 96 [Medium]

Key idea

Count structurally unique BSTs with values 1..n. Choose root i: left has i-1 nodes, right has n-i nodes -> multiply counts. This is the **Catalan number**.

Approach

- dp[0] = dp[1] = 1.
- dp[n] = sum over root i = 1..n of dp[i-1] * dp[n-i].
- Closed form: C(n) = (2n)! / ((n+1)! n!). Sequence: 1, 1, 2, 5, 14, 42, 132...

Code (C++)

```
int numTrees(int n) {
    vector<int> dp(n + 1, 0);
    dp[0] = 1;
    for (int nodes = 1; nodes <= n; nodes++)
        for (int root = 1; root <= nodes; root++)
            dp[nodes] += dp[root - 1] * dp[nodes - root];
    return dp[n];
}
```

Complexity: Time O(n^2), Space O(n).

Pitfalls / follow-ups

- Catalan also counts: valid parentheses strings, ways to triangulate a polygon, mountain ranges - a favourite aptitude/MCQ fact.
- Count of **binary trees** (labeled) with n nodes = Catalan(n) * n!.

22. Unique Binary Search Trees II LC 95 [Medium]

Key idea

Generate the actual trees. For each root i in [l, r], recursively generate all left trees from [l, i-1] and right trees from [i+1, r], and combine every pair.

Approach

- Base: l > r -> return {nullptr} (one empty tree, NOT an empty list).
- Compute left and right lists once per root, then nested loop.

Code (C++)

```
vector<TreeNode*> build(int l, int r) {
    if (l > r) return {nullptr};
    vector<TreeNode*> res;
    for (int i = l; i <= r; i++) {
        vector<TreeNode*> lefts = build(l, i - 1);
        vector<TreeNode*> rights = build(i + 1, r);
        for (TreeNode* L : lefts)
            for (TreeNode* R : rights)
                res.push_back(new TreeNode(i, L, R));
    }
    return res;
}
vector<TreeNode*> generateTrees(int n) { return build(1, n); }
```

Complexity: Time/Space about O(n * Catalan(n)) - output size dominates.

Pitfalls / follow-ups

- Returning `{}` instead of `{nullptr}` for the empty case gives zero trees - most common bug.
- Subtrees are shared between results (fine on LeetCode). Memoize on (l, r) to speed up.

23. Maximum Sum BST in Binary Tree LC 1373 [Hard]

Key idea

Find the max sum of any subtree that is a valid BST. Use **postorder** (bottom-up): each node returns {isBST, min, max, sum} of its subtree.

Approach

- Node forms a BST iff both children are BSTs and `left.max < val < right.min`.
- Null returns {true, +INF, -INF, 0} so the comparisons always pass.
- If valid: update answer with sum; else mark invalid (parents will also be invalid).

Code (C++)

```
struct Info { bool bst; int mn, mx, sum; };
int ans = 0;
Info dfs(TreeNode* n) {
    if (!n) return {true, INT_MAX, INT_MIN, 0};
    Info L = dfs(n->left), R = dfs(n->right);
    if (L.bst && R.bst && L.mx < n->val && n->val < R.mn) {
        int s = L.sum + R.sum + n->val;
        ans = max(ans, s);
        return {true, min(L.mn, n->val), max(R.mx, n->val), s};
    }
    return {false, 0, 0, 0};
}
int maxSumBST(TreeNode* root) { dfs(root); return ans; }
```

Complexity: Time O(n), Space O(h).

Pitfalls / follow-ups

- Answer starts at 0 - an empty BST is allowed, so all-negative trees give 0.
- Same template solves **'Largest BST in a Binary Tree'** (return size instead of sum) - very common on GFG/interviews.
- Top-down validation of every subtree would be O(n^2); bottom-up info passing makes it O(n).

24. Morris Traversal (practice) Practice [Concept]

Key idea

Inorder in **O(1) extra space** - no stack, no recursion. Temporarily make the inorder predecessor's right pointer point back to the current node (a 'thread'), then remove it.

Approach

- No left child: visit cur, go right.
- Has left child: find predecessor `pre` = rightmost node of cur->left.
- If `pre->right == null`: create thread `pre->right = cur`, go left.
- If `pre->right == cur`: left side is done; remove thread, visit cur, go right.
- Preorder version: visit cur when **creating** the thread instead of when removing it.

Code (C++)

```
vector<int> morrisInorder(TreeNode* root) {
    vector<int> res;
    TreeNode* cur = root;
    while (cur) {
        if (!cur->left) {
            res.push_back(cur->val);
            cur = cur->right;
        } else {
            TreeNode* pre = cur->left;
            while (pre->right && pre->right != cur) pre = pre->right;
            if (!pre->right) {            // create thread
                pre->right = cur;
                cur = cur->left;
            } else {                      // thread exists: left done
                pre->right = nullptr;
                res.push_back(cur->val);
                cur = cur->right;
            }
        }
    }
    return res;
}
```

Complexity: Time O(n) (each edge walked at most ~3 times), Space O(1).

Pitfalls / follow-ups

- Tree is temporarily modified but fully restored at the end.
- Practice by re-solving 94, 230, 530 and 99 with Morris - that is the usual O(1)-space follow-up.
- Also asked: LC 114 Flatten Binary Tree to Linked List uses the same 'rightmost of left subtree' idea.

Last-Minute Revision Sheet

Problem	One-line trick	Time / Space
94 Inorder	Stack: push left spine, pop, go right	O(n) / O(h)
700 Search	Go left or right by comparison	O(h) / O(1)
701 Insert	New value becomes a leaf	O(h) / O(h)
530 Min Diff	Min of adjacent inorder values	O(n) / O(h)
938 Range Sum	Prune: < low go right, > high go left	O(n) / O(h)
98 Validate	Pass (lo, hi), use long long	O(n) / O(h)
235 LCA	First split point of p and q	O(h) / O(1)
653 Two Sum IV	Inorder + two pointers (or hash set)	O(n) / O(n)
669 Trim	Out of range -> return trimmed other side	O(n) / O(h)
2476 Closest Queries	Inorder array + lower_bound	O(n + Q log n)
230 Kth Smallest	Iterative inorder, stop at k	O(h + k) / O(h)
450 Delete	0/1 child: return child; 2: copy successor	O(h) / O(h)
108 Array to BST	Middle element as root	O(n) / O(log n)
1382 Balance	Inorder nodes + rebuild by middle	O(n) / O(n)
173 Iterator	Stack of left spine, amortized O(1)	O(1) / O(h)
285 Successor	Going left -> record candidate	O(h) / O(1)
703 Kth Largest	Min-heap of size k	O(log k) / O(k)
1008 From Preorder	Global index + upper bound	O(n) / O(h)
99 Recover	First drop: prev; last drop: cur; swap	O(n) / O(h)
538 Greater Tree	Reverse inorder + running sum	O(n) / O(h)
96 Unique BSTs	dp[n] = sum dp[i-1]*dp[n-i] (Catalan)	O(n^2) / O(n)
95 Unique BSTs II	Every root x all left x all right, base {nullptr}	O(n Cat(n))
1373 Max Sum BST	Postorder {isBST, min, max, sum}	O(n) / O(h)
Morris	Thread predecessor->right to cur	O(n) / O(1)

Interview tips

- Always state: 'Inorder of a BST is sorted' - then check if the problem becomes an array problem.
- Mention complexity in terms of **h**, then clarify balanced (log n) vs skewed (n).
- Ask about duplicates and value ranges (INT_MIN / INT_MAX edge cases in Validate BST).
- Have a ready answer for O(1) space follow-ups: Morris traversal.
- Commonly paired theory questions: BST vs Heap vs Hash table, AVL vs Red-Black, why std::map is O(log n).